AUTOSCALING IN KUBERNETES

WHY WE NEED PODS AUTOSCALING

1. if suppose i have a pod and these pods utilize all the resources of a node then we must create a node
2. this thing happens or creating nodes we can generally add limitations on the pod
3. means if the pod utilizes the 70% resources of a node after that it can automatically create a pod of a existing configuration That also having the limitation of 70% resources of the node.

TYPES OF AUTOSCALING

1. Horizontal Pod autoscaling

In horizontal pod autoscaling we can scale up and scale down the pod depends on the CPU, Memory and the custom metrics.

2. Vertical Pod autoscaling

In vertical pod autoscaling we can add the resources on the existing pod like the CPU, Memory means restarts the pod.

3. Cluster autoscaling

In cluster autoscaling we simply scale the nodes means we can add or remove the nodes automatically.

HORIZONTAL POD AUTOSCALING

1.vim namespace.yml

```
apiVersion: v1
kind: Namespace
metadata:
  name: apache
```

2.vim deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
```

```
metadata:
  name: apache-deployment
  labels:
    app: apache
    env: devops
  namespace: apache
spec:
  replicas: 1
  selector:
    matchLabels:
      app: apache
  template:
    metadata:
      name: my-apache-pod
      labels:
        app: apache
    spec:
      containers:
      - name: apache-container
        image: httpd:latest
        ports:
        - containerPort: 80
        resources:
          requests:
            cpu: 100m
            memory: 128Mi
          limits:
            cpu: 200m
            memory: 256Mi
```

3.vim service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: apache-service
  labels:
    app: apache
    env: devops
```

```
  namespace: apache
spec:
  type: ClusterIP
  selector:
    app: apache
  ports:
  - name: apache-port
    protocol: TCP
    port: 80
    targetPort: 80
```

4.vim hpauto.yaml

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: apache-hpa
  namespace: apache
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: apache-deployment
  minReplicas: 1
  maxReplicas: 3
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
```

1. kubectl apply -f namespace.yaml
2. kubectl apply -f deployment.yaml
3. kubectl apply -f service.yaml
4. kubectl apply -f hpauto.yaml
5. kubectl get all -n apache

INSTALL METRICS SERVER IN EKS CLUSTER

1. kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
2. kubectl get pods -n kube-system
3. kubectl top pods
4. kubectl top nodes

ERROR:

1. If u get error of not running the pods
2. edit the deployment metrics-sever

kubectl edit deployment metrics-server -n kube-system
In args: section add these line
- --kubelet-insecure-tls

3. Restart the deployment
    1. kubectl rollout restart deployment metrics-server -n kube-system
    2. kubectl rollout status deployment metrics-server -n kube-system

---

TO QUICKLY ADD THE STRESS ON THE PODS

1. kubectl exec -it <pod_name> -n apache -- bash
2. while true; do :; done

CREATE A DUPLICATE TERMINAL TO CHECK THE PODS

1. kubectl get hpa -n apache
2. kubectl get pods -n apache
3. watch kubectl get pods -n apache

VERTICAL POD AUTOSCALING

1. git clone https://github.com/kubernetes/autoscaler.git
2. cd autoscaler/vertical-pod-autoscaler/
3. ./hack/vpa-up.sh
4. cd

5. mkdir vpa
6. cd vpa

7. **Vim namespace.yaml**

```
apiVersion: v1
kind: Namespace
metadata:
  name: apache
```

8. **Vim deployment.yaml**

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: apache-deployment
  labels:
    app: apache
    env: devops
  namespace: apache
spec:
  replicas: 1
  selector:
    matchLabels:
      app: apache
  template:
    metadata:
      name: my-apache-pod
      labels:
        app: apache
    spec:
      containers:
      - name: apache
        image: httpd:latest
```

```
        ports:
        - containerPort: 80
        resources:
          requests:
            cpu: 100m
            memory: 128Mi
          limits:
            cpu: 200m
            memory: 256Mi
```

9. Vim service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: apache-service
  labels:
    app: apache
    env: devops
  namespace: apache
spec:
  type: ClusterIP
  selector:
    app: apache
  ports:
  - name: apache-port
    protocol: TCP
    port: 80
    targetPort: 80
```

10. Vim vpa.yaml

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: apache-vpa
  namespace: apache
spec:
  targetRef:
    name: apache-deployment
    apiVersion: apps/v1
```

```
    kind: Deployment
  updatePolicy:
    updateMode: "Auto"
```

11. Kubectl apply -f .
12. Kubectl get all -n apache

ADD STRESS ON THE POD

13. while true; do :; done

14. watch kubectl get vpa -n apache

TAINTS AND TOLERANCE

1. Taint is applied on a node to prevent pods from being scheduled on that node.
2. The Kubernetes scheduler avoids placing pods on tainted nodes unless the pod can tolerate the taint.
3. Toleration is applied to a pod to allow it to be scheduled on a tainted node.
4. Taints control node behavior, while tolerations control pod behavior.

TAINTS

1. **To taint the nodes**
    1. kubectl taint node worker-node1 prod=true:NoSchedule
    2. kubectl taint node worker-node2 prod=true:NoSchedule

2. **Run a pod**
    1. kubectl run mypod --image=nginx --port=80
       {pod is in the pending state}

3. **To remove the taint (untainted)**
    1. kubectl taint node worker-node1 prod=true:NoSchedule-
       {automatically pod schedule on the worker-node-1}

TOLERANCE

1. vim tolerance.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: mypod
  labels:
```

```
    app: myapp
    env: devops
spec:
  tolerations:
    - key: "prod"
      operator: "Equal"
      value: "true"
      effect: "NoSchedule"
  containers:
    - name: nginx
      image: nginx:latest
      ports:
        - name: myport
          containerPort: 80
```

1. kubectl apply -f pod.yml
2. kubectl get pods -o wide

NOTE

1. If all worker nodes are tainted, pods go into Pending state (unless they have tolerations).
2. Control-plane nodes are already tainted (to prevent normal pods from scheduling there).
3. Tolerations don't guarantee scheduling — they just allow it if no other restrictions exist (like node selectors or affinity rules).

NODE SELECTOR

Node Selector is used to schedule a pod on a specific node based on node labels.

1. kubectl label node worker-node type=frontend
2. kubectl get nodes --show-labels

3. vim pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  nodeSelector:
```

```
    type: frontend
  containers:
    - name: mycontainer
      image: nginx:latest
      ports:
        - name: myport
          containerPort: 80
```

4. kubectl apply -f pod.yaml
5. kubectl get pods -o wide

NODE-AFFINITY

1. Node Affinity is used to control which node a pod can be scheduled on based on node labels.
2. It is a more powerful and flexible version of Node Selector.
3. It is generally preferred in production.

PRACTICAL

1. kubectl label node worker-node type=frontend
2. kubectl get nodes --show-labels

3. vim pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
  labels:
    app: myapp
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: type
            operator: In
            values:
            - frontend
  containers:
```

```
  - name: httpd-container
    image: httpd:latest
    ports:
    - containerPort: 80
```

4. kubectl apply -f pod.yaml
5. kubectl get pods -o wide

TYPES OF NODE AFFINITY

1. requiredDuringSchedulingIgnoredDuringExecution (mandatory)
    1. The scheduler must find a matching node.
    2. If no matching node exists, the pod remains in Pending state.
    3. Schedule this pod only on nodes where type=frontend.

2. preferredDuringSchedulingIgnoredDuringExecution (Preferred)
    1. The scheduler tries to place the pod on matching nodes.
    2. If no matching node exists, it schedules the pod elsewhere.
    3. Prefer frontend nodes, but if none are available, use another node.

Note:

1. it also supports the multiple operators like (In / NotIn / exists / DoesNotExists)
